

Ruby para quem sabe Python

Você já sabe programar. Isto aqui é tradução, não curso de lógica.

Sintaxe

Python	Ruby
Indentação delimita bloco	<code>def ... end</code>
<code>None</code> / <code>True</code> / <code>False</code>	<code>nil</code> / <code>true</code> / <code>false</code>
<code>snake_case</code> para funções	<code>snake_case</code> (igual)
<code>PascalCase</code> para classes	<code>PascalCase</code> (igual)
<code>CONSTANTE</code> por convenção	Constante é qualquer nome com inicial maiúscula
<code>f"Olá {nome}"</code>	<code>"Olá #{nome}"</code>
<code># comentário</code>	<code># comentário</code>
<code>return</code> explícito	A última expressão é o retorno
<code>and</code> / <code>or</code> / <code>not</code>	<code>&&</code> / <code>& \ </code> / <code>!</code>
<code>elif</code>	<code>elsif</code>
<code>len(x)</code>	<code>x.length</code>
<code>str(x)</code>	<code>x.to_s</code>
<code>int("3")</code>	<code>"3".to_i</code>
<code>print(x)</code>	<code>puts x</code>

Só em Ruby

```
faca_algo if condicao          # modificador: if no fim da linha
faca_algo unless condicao      # unless = if not
5.times { |i| puts i }        # número é objeto
nome = usuario&.nome          # safe navigation (o ?. de outras linguagens)
valor ||= "padrao"            # atribui só se estiver nil/false
```

Aspas: a diferença que Python não tem

Em Python, `'a'` e `"a"` são a mesma coisa. **Em Ruby, não:**

```
"Olá, #{nome}"      # interpola
'Olá, #{nome}'      # imprime literalmente #{nome}
```

Aspas simples são texto cru. Use aspas duplas por padrão; simples quando o texto tiver `#{` ou muitas barras invertidas.

Números: a pegadinha da divisão

```
7 / 2      # 3.5    em Python 3
7 // 2     # 3
```

```
7 / 2      # 3      - inteiro dividido por inteiro dá inteiro
7.0 / 2    # 3.5
7.fdiv(2)  # 3.5
```

Python 3 mudou esse comportamento; Ruby não. É a origem de um bug silencioso quando se calcula média ou porcentagem.

Condicionais e `case`

```
# Python
if x == 1:
    ...
elif x == 2:
    ...
else:
    ...

match x:                                # Python 3.10+
    case 1: ...
    case 2: ...
```

```
# Ruby
if x == 1
    ...
elsif x == 2
    ...
else
    ...
end

case x
when 1 then ...
when 2 then ...
else ...
end
```

O `then` permite a linha única. O `case` de Ruby também aceita faixa, classe e regex:

```
case idade
when 0..12 then "criança"
when 13..17 then "adolescente"
else
    "adulto"
end
```

E, ao contrário de `switch` em C ou Java, **não existe** `break`: cada `when` já para sozinho.

Isso aparece no `SessionsController`:

```
case session_params[:client]
when "mobile" then response.headers["Authorization"] = "Bearer #{token}"
when "web"     then set_auth_cookie(token)
end
```

Ternário e loops

```
mensagem = ativo ? "sim" : "não"      # em Python: "sim" if ativo else "não"

while restam > 0 ... end
until pronto ... end                  # o contrário do while – não existe em Python
loop do ... break if pronto ... end
```

`puts`, `print` e `p`

Ruby	Faz
<code>puts x</code>	imprime com quebra de linha. É o <code>print()</code> do dia a dia.
<code>print x</code>	imprime sem quebra de linha
<code>p x</code>	imprime a representação (com aspas, colchetes) e devolve o objeto

`p` é o que você usa para depurar: `p usuario` mostra o objeto inteiro, e como ele devolve o valor, dá para enfiar no meio de uma expressão sem quebrar nada.

`require` × `import`

```
import json                # Python
from pathlib import Path
```

```
require "json"             # Ruby: biblioteca ou gem
require_relative "helper"  # arquivo ao lado, caminho relativo
```

`require` carrega uma vez só e devolve `false` se já estava carregado. **Não** traz nomes para o escopo como o `from x import y`: depois do `require`, você usa `JSON.parse` com o nome completo.

Dentro do Rails você nunca escreve `require`. O Zeitwerk carrega a classe pelo nome do arquivo. Fora do Rails, num script solto, o `require` volta a ser necessário.

Splat

```
def f(*args, **kwargs): ...
```

```
def f(*args, **opts); end
```

```
valores = [1, 2, 3]
```

```
f(*valores) # espalha o array nos argumentos
```

Estruturas de dados

Python	Ruby
<code>[1, 2, 3]</code>	<code>[1, 2, 3]: Array</code>
<code>{"a": 1}</code>	<code>{ a: 1 }: Hash</code> , chave símbolo
<code>{"a": 1}["a"]</code>	<code>{ a: 1 }[:a]</code>
<code>(1, 2)</code> tupla	não existe (use array congelado)
<code>{1, 2}</code> set	<code>Set.new([1, 2])</code>
não tem	<code>:simbolo</code>

```
usuario = { nome: "Ana", role: :admin }  
usuario[:nome] # "Ana"  
usuario.fetch(:idade, 0) # 0 – como dict.get com padrão
```

Símbolos

Um símbolo é uma **etiqueta**: um texto usado como nome, não como conteúdo. O mesmo símbolo é sempre o mesmo objeto na memória, o que o torna barato de comparar.

```
"admin" == "admin" # true, mas são dois objetos diferentes  
:admin == :admin # true, e é o mesmo objeto
```

Regra prática: conteúdo que o usuário lê → string. Nome de coisa no código → símbolo.

Blocos, o coração do Ruby

Um bloco é código que você entrega para um método executar. Em Python isso é o `lambda`, usado de vez em quando. Em Ruby é o caminho normal.

```
# Python
for u in usuarios:
    print(u.nome)

nomes = [u.nome for u in usuarios]
ativos = [u for u in usuarios if u.ativo]
total = sum(u.pontos for u in usuarios)
```

```
# Ruby
usuarios.each do |u|
    puts u.nome
end

nomes = usuarios.map { |u| u.nome }
ativos = usuarios.select { |u| u.ativo? }
total = usuarios.sum { |u| u.pontos }
```

Chaves quando cabe numa linha, `do ... end` quando não cabe.

Bloco, proc e lambda

Bloco não é objeto: ele existe só na chamada do método. Quando você quer **guardar** o código numa variável ou passar adiante, ele vira `Proc` ou `lambda`.

```
dobro = lambda x: x * 2      # Python
dobro(3)
```

```
dobro = ->(x) { x * 2 }      # Ruby: a "seta" é um lambda
dobro.call(3)
dobro.(3)                    # açúcar
```

A seta `->` é o que você vai ver no código deste projeto:

```
scope :recentes, -> { order(occurred_at: :desc) }
validate :password_meets_policy, if: -> { password.present? }
```

Nos dois casos o Rails guarda aquele pedaço de código para executar **depois**: na hora da consulta ou na hora da validação. Por isso precisa ser um objeto, e não um bloco.

`->` e `Proc.new` são quase a mesma coisa. A diferença que importa: o `lambda` confere o número de argumentos e o `return` dele volta só do `lambda`. O `Proc` é relaxado nos dois pontos. Na dúvida, use `->`.

Equivalências

Python	Ruby
list comprehension	<code>.map</code>
<code>filter</code> / comprehension com <code>if</code>	<code>.select</code> (e <code>.reject</code> para o inverso)
<code>functools.reduce</code>	<code>.reduce</code> / <code>.inject</code>
<code>any()</code> / <code>all()</code>	<code>.any?</code> / <code>.all?</code>
<code>sorted(x, key=...)</code>	<code>x.sort_by { ... }</code>
<code>enumerate</code>	<code>.each_with_index</code>
<code>zip</code>	<code>.zip</code>
<code>next(x for x in l if ...)</code>	<code>.find { ... }</code>

Argumentos nomeados

```
# Python
def login(email, password, client="mobile"): ...
login(email="a@b.c", password="x")
```

```
# Ruby – os dois-pontos DEPOIS do nome tornam obrigatório nomear na chamada
def login(email:, password:, client: "mobile")
  end

login(email: "a@b.c", password: "x")
```

Sem os dois-pontos, é argumento posicional como em Python. Com eles, quem chama **tem** que nomear, e a ordem deixa de importar.

O atalho do Ruby 3.1

Quando a variável tem o mesmo nome da chave, você omite o valor:

```
email = "a@b.c"
password = "x"

login(email:, password:)      # em vez de login(email: email, password: password)
```

Isso aparece em **todo service do projeto**:

```
Result.new(success?: true, user:)    # user: user
```

Não é erro de digitação.

Struct

Quando você só quer agrupar valores, sem comportamento:

```
Result = Struct.new(:success?, :user, :errors, keyword_init: true)

r = Result.new(success?: true, user: ana)
r.success?    # true
r.user        # ana
```

`keyword_init: true` obriga a nomear na criação. É o `namedtuple` / `dataclass` do Python. Todo service deste projeto devolve um `Struct` desses.

Exceções

Python	Ruby
<code>try</code>	<code>begin</code>
<code>except</code>	<code>rescue</code>
<code>finally</code>	<code>ensure</code>
<code>raise</code>	<code>raise</code>
<code>Exception</code> (base para capturar)	<code>StandardError</code>

```
try:
    arriscado()
except ValueError as e:
    print(e)
finally:
    limpar()
```

```
begin
  arriscado
rescue ArgumentError => e
  puts e.message
ensure
  limpar
end
```

Dentro de um método, o `begin` é implícito. Dá para escrever só o `rescue` no fim:

```
def call
  arriscado
rescue StandardError => e
  Rails.error.report(e)
  nil
end
```

Criando o seu tipo de erro:

```
class InvalidToken < StandardError; end

raise InvalidToken if token_ruim?
```

Herde sempre de `StandardError`, nunca de `Exception`. `rescue` sem classe captura `StandardError`; capturar `Exception` pega até `Ctrl+C` e erro de falta de memória.

`raise` sem argumento, dentro de um `rescue`, relança a exceção atual.

Atalhos de literal

```
%w[mobile web]      # => ["mobile", "web"]    – array de strings
%i[name email]      # => [:name, :email]    – array de símbolos
```

Só economizam aspas e vírgulas. Aparecem em constantes por todo o projeto:

```
CLIENTS = %w[mobile web].freeze
REQUIRED_FIELDS = %i[name email password].freeze
```

`.freeze` congela o objeto: tentar alterar depois levanta erro. Em Ruby strings e arrays são mutáveis (diferente de Python), então constante sem `freeze` é constante só no nome.

Classes

```
class Usuario:
    def __init__(self, nome):
        self.nome = nome

    def saudacao(self):
        return f"Oi, {self.nome}"

    @property
    def admin(self):
        return self.role == "admin"
```

```
class Usuario
  attr_accessor :nome          # gera o getter e o setter

  def initialize(nome)
    @nome = nome              # @ marca variável de instância
  end

  def saudacao
    "Oi, #{@nome}"
  end

  def admin?
    role == "admin"
  end
end
```

Python	Ruby
<code>__init__</code>	<code>initialize</code>
<code>self.x</code>	<code>@x</code>
<code>self</code> como primeiro parâmetro	<code>self</code> implícito
<code>@property</code>	método normal (não precisa de parênteses)
<code>@staticmethod</code>	<code>def self.metodo</code>
<code>_privado</code> por convenção	<code>private</code> de verdade
herança múltipla	módulos (<code>include</code>)

? e !

```
user.admin?    # devolve true/false
user.save      # devolve false se falhar
user.save!     # estoura ActiveRecord::RecordInvalid se falhar
```

Convenção, não regra da linguagem. Mas todo mundo segue, e o Rails inteiro depende dela.

Módulos e mixins

Ruby não tem herança múltipla. Tem módulo: um saco de métodos que você inclui.

```
# Python: mixin por herança múltipla
class Usuario(Base, VerificavelPorEmail, RecuperaSenha):
    pass
```

```
# Ruby: include
class Usuario < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

```
module EmailVerifiable
  extend ActiveSupport::Concern

  def email_verificado?
    email_verified_at.present?
  end
end
```

No Rails, um módulo desses vive em `app/models/concerns/` e se chama **concern**. É o que impede o `User` de virar um arquivo de 800 linhas.

Dependências e ferramentas

Python	Ruby
<code>pip install x</code>	<code>gem install x</code>
<code>requirements.txt</code>	<code>Gemfile</code>
<code>pip freeze > requirements.txt</code>	<code>Gemfile.lock</code> (gerado sozinho, sempre versionado)
<code>venv</code> / <code>virtualenv</code>	<code>bundler</code> isola por projeto
<code>pyenv</code>	<code>mise</code> / <code>rbenv</code>
<code>python -m x</code>	<code>bundle exec x</code>
<code>pytest</code>	<code>minitest</code> (padrão do Rails) ou <code>rspec</code>
<code>black</code> / <code>ruff</code>	<code>rubocop</code>
<code>mypy</code>	não há equivalente padrão: Ruby é dinâmica de ponta a ponta
<code>python</code> (REPL)	<code>irb</code>

Rails × Django

Django	Rails
<code>models.py</code> com classes <code>Model</code>	<code>app/models/</code> , um arquivo por classe
<code>makemigrations</code> / <code>migrate</code>	<code>rails g migration</code> / <code>rails db:migrate</code>
<code>urls.py</code> , escrito à mão	<code>config/routes.rb</code>
<code>views.py</code>	<code>app/controllers/</code>
<code>serializers.py</code> (DRF)	<code>app/serializers/</code> (feito à mão neste projeto)
Django ORM: <code>User.objects.filter(...)</code>	ActiveRecord: <code>User.where(...)</code>
<code>manage.py</code>	<code>bin/rails</code>
<code>settings.py</code>	<code>config/application.rb</code> + <code>config/environments/</code>
<code>.env</code> + <code>django-environ</code>	<code>Rails.application.credentials</code> + ENV
<code>python manage.py shell</code>	<code>bin/rails console</code>

Consultas lado a lado

```
User.objects.filter(role="admin")
User.objects.get(id=1)
User.objects.filter(email__icontains="ufop").order_by("-created_at")[:10]
User.objects.count()
```

```
User.where(role: "admin")
User.find(1)
User.where("email ILIKE ?", "%ufop%").order(created_at: :desc).limit(10)
User.count
```

Pegadinhas de quem vem do Python

Pegadinha	O que acontece
<code>0</code> e <code>""</code> são verdadeiros em Ruby	Só <code>nil</code> e <code>false</code> são falsos. <code>if 0</code> executa.
<code>puts</code> devolve <code>nil</code>	Não use o retorno de <code>puts</code>
Parênteses são opcionais	<code>user.save</code> e <code>user.save()</code> são a mesma coisa
<code>=</code> no fim do nome define setter	<code>def nome=(valor)</code> habilita <code>obj.nome = "x"</code>
Strings são mutáveis	Diferente de Python. Use <code>.freeze</code> em constantes.
<code>7 / 2</code> dá <code>3</code> , não <code>3.5</code>	Python 3 mudou isso; Ruby não. Use <code>7.0 / 2</code> ou <code>7.fdiv(2)</code> .
<code>'texto #{x}'</code> não interpola	Só aspas duplas interpolam.
<code>case</code> não precisa de <code>break</code>	Cada <code>when</code> já para sozinho.
<code>and</code> / <code>or</code> existem, mas têm precedência diferente de <code>&&</code> / <code>\ \ </code>	Use <code>&&</code> e <code>\ \ </code> sempre
Não existe <code>elif</code>	É <code>elsif</code>
Range com <code>..</code> inclui o fim, com <code>...</code> não	<code>(1..3).to_a</code> → <code>[1,2,3]</code> ; <code>(1...3).to_a</code> → <code>[1,2]</code>